FULL_EXPORT.md - Freedom Project Complete Documentation

Generated: Fri Apr 3 04:53:57 EDT 2026

1. PROJECT OVERVIEW

Freedom - Autonomous Debt Management Ecosystem

2. PACKAGE.JSON - Dependencies

```
{
  "name": "freedom-app",
  "version": "1.0.0",
  "type": "module",
  "license": "MIT",
  "scripts": {
    "dev": "NODE_ENV=development tsx watch server/_core/index.ts",
    "build": "vite build && esbuild server/_core/index.ts --platform=node --
packages=external --bundle --format=esm --outdir=dist",
    "start": "NODE_ENV=production node dist/index.js",
    "check": "tsc --noEmit",
    "format": "prettier --write .",
    "test": "vitest run",
    "db:push": "drizzle-kit generate && drizzle-kit migrate"
  },
  "dependencies": {
    "@aws-sdk/client-s3": "^3.693.0",
    "@aws-sdk/s3-request-presigner": "^3.693.0",
    "@hookform/resolvers": "^5.2.2",
    "@radix-ui/react-accordion": "^1.2.12",
    "@radix-ui/react-alert-dialog": "^1.1.15",
    "@radix-ui/react-aspect-ratio": "^1.1.7",
    "@radix-ui/react-avatar": "^1.1.10",
    "@radix-ui/react-checkbox": "^1.3.3",
    "@radix-ui/react-collapsible": "^1.1.12",
    "@radix-ui/react-context-menu": "^2.2.16",
    "@radix-ui/react-dialog": "^1.1.15",
    "@radix-ui/react-dropdown-menu": "^2.1.16",
    "@radix-ui/react-hover-card": "^1.1.15",
    "@radix-ui/react-label": "^2.1.7",
    "@radix-ui/react-menubar": "^1.1.16",
    "@radix-ui/react-navigation-menu": "^1.2.14",
    "@radix-ui/react-popover": "^1.1.15",
    "@radix-ui/react-progress": "^1.1.7",
    "@radix-ui/react-radio-group": "^1.3.8",
    "@radix-ui/react-scroll-area": "^1.2.10",
    "@radix-ui/react-select": "^2.2.6",
    "@radix-ui/react-separator": "^1.1.7",
    "@radix-ui/react-slider": "^1.3.6",
    "@radix-ui/react-slot": "^1.2.3",
    "@radix-ui/react-switch": "^1.2.6",
    "@radix-ui/react-tabs": "^1.1.13",
```

```
    "@radix-ui/react-toggle": "^1.1.10",
    "@radix-ui/react-toggle-group": "^1.1.11",
    "@radix-ui/react-tooltip": "^1.2.8",
    "@tanstack/react-query": "^5.90.2",
    "@trpc/client": "^11.6.0",
    "@trpc/react-query": "^11.6.0",
    "@trpc/server": "^11.6.0",
    "axios": "^1.12.0",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "cmdk": "^1.1.1",
    "cookie": "^1.0.2",
    "date-fns": "^4.1.0",
    "dotenv": "^17.2.2",
    "drizzle-orm": "^0.44.5",
    "embla-carousel-react": "^8.6.0",
    "express": "^4.21.2",
    "framer-motion": "^12.23.22",
    "input-otp": "^1.4.2",
    "jose": "6.1.0",
    "lucide-react": "^0.453.0",
    "mysql2": "^3.15.0",
    "nanoid": "^5.1.5",
    "next-themes": "^0.4.6",
    "react": "^19.2.1",
    "react-day-picker": "^9.11.1",
    "react-dom": "^19.2.1",
    "react-hook-form": "^7.64.0",
    "react-resizable-panels": "^3.0.6",
    "recharts": "^2.15.2",
    "sonner": "^2.0.7",
    "streamdown": "^1.4.0",
    "superjson": "^1.13.3",
    "tailwind-merge": "^3.3.1",
    "tailwindcss-animate": "^1.0.7",
    "vaul": "^1.1.2",
    "wouter": "^3.3.5",
    "zod": "^4.1.12"
  },
  "devDependencies": {
    "@builder.io/vite-plugin-jsx-loc": "^0.1.1",
    "@tailwindcss/typography": "^0.5.15",
    "@tailwindcss/vite": "^4.1.3",
    "@types/express": "4.17.21",
    "@types/google.maps": "^3.58.1",
    "@types/node": "^24.7.0",
```

```
    "@types/react": "^19.2.1",
    "@types/react-dom": "^19.2.1",
    "@vitejs/plugin-react": "^5.0.4",
    "add": "^2.0.6",
    "autoprefixer": "^10.4.20",
    "drizzle-kit": "^0.31.4",
    "esbuild": "^0.25.0",
    "pnpm": "^10.15.1",
    "postcss": "^8.4.47",
    "prettier": "^3.6.2",
    "tailwindcss": "^4.1.14",
    "tsx": "^4.19.1",
    "tw-animate-css": "^1.4.0",
    "typescript": "5.9.3",
    "vite": "^7.1.7",
    "vite-plugin-manus-runtime": "^0.0.57",
    "vitest": "^2.1.4"
  },
  "packageManager":
"pnpm@10.4.1+sha512.c753b6c3ad7afa13af388fa6d808035a008e30ea9993f58c6663e2bc5f
  "pnpm": {
    "patchedDependencies": {
      "wouter@3.7.1": "patches/wouter@3.7.1.patch"
    },
    "overrides": {
      "tailwindcss>nanoid": "3.3.7"
    }
  }
}```

## 3. DATABASE SCHEMA

```typescript
import { int, mysqlEnum, mysqlTable, text, timestamp, varchar, boolean,
decimal, json, longtext } from "drizzle-orm/mysql-core";
import { relations } from "drizzle-orm";

/**
 * Core user table backing auth flow.
 * Extend this file with additional tables as your product grows.
 * Columns use camelCase to match both database fields and generated types.
 */
export const users = mysqlTable("users", {
  id: int("id").autoincrement().primaryKey(),
  openId: varchar("openId", { length: 64 }).notNull().unique(),
  name: text("name"),
```
```

```
  email: varchar("email", { length: 320 }),
  phoneNumber: varchar("phoneNumber", { length: 20 }),
  loginMethod: varchar("loginMethod", { length: 64 }),
  role: mysqlEnum("role", ["debtor", "professional",
"admin"]).default("debtor").notNull(),
  userType: mysqlEnum("userType", ["individual",
"professional"]).default("individual").notNull(),
  webauthnCredentials: longtext("webauthnCredentials"), // JSON array of
WebAuthn credentials
  twoFactorEnabled: boolean("twoFactorEnabled").default(false),
  twoFactorSecret: varchar("twoFactorSecret", { length: 255 }),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
  lastSignedIn: timestamp("lastSignedIn").defaultNow().notNull(),
});

export type User = typeof users.$inferSelect;
export type InsertUser = typeof users.$inferInsert;

// Override InsertUser to make userType optional for backward compatibility
export type InsertUserRequest = Omit<InsertUser, 'userType'> & { userType?:
InsertUser['userType'] };

// Professional profiles table
export const professionalProfiles = mysqlTable("professional_profiles", {
  id: int("id").autoincrement().primaryKey(),
  userId: int("userId").notNull(),
  specializations: json("specializations"), // ['lawyer', 'accountant',
'financial_advisor']
  licenseNumber: varchar("licenseNumber", { length: 255 }),
  yearsOfExperience: int("yearsOfExperience"),
  bio: text("bio"),
  hourlyRate: decimal("hourlyRate", { precision: 10, scale: 2 }),
  isVerified: boolean("isVerified").default(false),
  verificationDate: timestamp("verificationDate"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
});

export type ProfessionalProfile = typeof professionalProfiles.$inferSelect;
export type InsertProfessionalProfile = typeof
professionalProfiles.$inferInsert;

// Debt profiles / cases table
export const debtProfiles = mysqlTable("debt_profiles", {
  id: int("id").autoincrement().primaryKey(),
```

```
  userId: int("userId").notNull(),
  totalDebtAmount: decimal("totalDebtAmount", { precision: 15, scale: 2
}).notNull(),
  debtType: mysqlEnum("debtType", ["bank", "credit_card", "personal_loan",
"mortgage", "tax", "other"]).notNull(),
  severity: mysqlEnum("severity", ["low", "medium", "high",
"critical"]).notNull(),
  persona: mysqlEnum("persona", ["yossi", "dana", "avi"]).notNull(), //
Yossi (early), Dana (advanced), Avi (critical)
  status: mysqlEnum("status", ["new", "in_progress", "resolved",
"archived"]).default("new"),
  triageCompletedAt: timestamp("triageCompletedAt"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
});

export type DebtProfile = typeof debtProfiles.$inferSelect;
export type InsertDebtProfile = typeof debtProfiles.$inferInsert;

// Cases table (linking debtors to professionals)
export const cases = mysqlTable("cases", {
  id: int("id").autoincrement().primaryKey(),
  debtProfileId: int("debtProfileId").notNull(),
  professionalUserId: int("professionalUserId"),
  status: mysqlEnum("status", ["open", "in_progress", "closed",
"on_hold"]).default("open"),
  matchingScore: decimal("matchingScore", { precision: 5, scale: 2 }),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
});

export type Case = typeof cases.$inferSelect;
export type InsertCase = typeof cases.$inferInsert;

// Documents table with encryption metadata
export const documents = mysqlTable("documents", {
  id: int("id").autoincrement().primaryKey(),
  caseId: int("caseId").notNull(),
  uploadedByUserId: int("uploadedByUserId").notNull(),
  fileName: varchar("fileName", { length: 255 }).notNull(),
  fileKey: varchar("fileKey", { length: 255 }).notNull(), // S3 key
  fileUrl: text("fileUrl").notNull(), // Encrypted S3 URL
  mimeType: varchar("mimeType", { length: 100 }),
  fileSizeBytes: int("fileSizeBytes"),
  encryptionAlgorithm: varchar("encryptionAlgorithm", { length: 50
}).default("AES-256"),
```

```
  encryptionKeyId: varchar("encryptionKeyId", { length: 255 }),
  documentType: mysqlEnum("documentType", ["contract", "statement",
"letter", "agreement", "other"]),
  ocrProcessed: boolean("ocrProcessed").default(false),
  extractedText: longtext("extractedText"), // Encrypted extracted text from
OCR
  aiSummary: longtext("aiSummary"), // Encrypted AI summary
  extractedTasks: json("extractedTasks"), // Encrypted JSON array of
extracted tasks
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
});

export type Document = typeof documents.$inferSelect;
export type InsertDocument = typeof documents.$inferInsert;

// Tasks table
export const tasks = mysqlTable("tasks", {
  id: int("id").autoincrement().primaryKey(),
  caseId: int("caseId").notNull(),
  assignedToUserId: int("assignedToUserId").notNull(),
  title: varchar("title", { length: 255 }).notNull(),
  description: text("description"),
  status: mysqlEnum("status", ["pending", "in_progress", "completed",
"blocked"]).default("pending"),
  priority: mysqlEnum("priority", ["low", "medium", "high",
"urgent"]).default("medium"),
  dueDate: timestamp("dueDate"),
  completedAt: timestamp("completedAt"),
  createdByUserId: int("createdByUserId").notNull(),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
});

export type Task = typeof tasks.$inferSelect;
export type InsertTask = typeof tasks.$inferInsert;

// Consent records table (Privacy Law Amendment 13 compliance)
export const consentRecords = mysqlTable("consent_records", {
  id: int("id").autoincrement().primaryKey(),
  userId: int("userId").notNull(),
  consentType: mysqlEnum("consentType", ["data_processing", "ai_analysis",
"professional_sharing", "communication"]).notNull(),
  consentGiven: boolean("consentGiven").notNull(),
  consentText: text("consentText"),
  ipAddress: varchar("ipAddress", { length: 45 }),
```

```
  userAgent: text("userAgent"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  expiresAt: timestamp("expiresAt"),
});

export type ConsentRecord = typeof consentRecords.$inferSelect;
export type InsertConsentRecord = typeof consentRecords.$inferInsert;

// Audit logs table
export const auditLogs = mysqlTable("audit_logs", {
  id: int("id").autoincrement().primaryKey(),
  userId: int("userId"),
  actionType: varchar("actionType", { length: 100 }).notNull(),
  resourceType: varchar("resourceType", { length: 100 }),
  resourceId: varchar("resourceId", { length: 255 }),
  actionDetails: json("actionDetails"), // Non-sensitive operation metadata
  ipAddress: varchar("ipAddress", { length: 45 }),
  userAgent: text("userAgent"),
  result: mysqlEnum("result", ["success", "failure", "denied"]).notNull(),
  errorMessage: text("errorMessage"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});

export type AuditLog = typeof auditLogs.$inferSelect;
export type InsertAuditLog = typeof auditLogs.$inferInsert;

// Notifications table
export const notifications = mysqlTable("notifications", {
  id: int("id").autoincrement().primaryKey(),
  userId: int("userId").notNull(),
  notificationType: mysqlEnum("notificationType", ["task_assigned",
"document_uploaded", "case_update", "reminder", "system"]).notNull(),
  title: varchar("title", { length: 255 }).notNull(),
  message: text("message"),
  relatedCaseId: int("relatedCaseId"),
  relatedTaskId: int("relatedTaskId"),
  isRead: boolean("isRead").default(false),
  readAt: timestamp("readAt"),
  deliveryChannel: mysqlEnum("deliveryChannel", ["in_app", "email",
"whatsapp"]).default("in_app"),
  deliveryStatus: mysqlEnum("deliveryStatus", ["pending", "sent",
"delivered", "failed"]).default("pending"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});

export type Notification = typeof notifications.$inferSelect;
```

```
export type InsertNotification = typeof notifications.$inferInsert;

// Diagnoses table
export const diagnoses = mysqlTable("diagnoses", {
  id: int("id").autoincrement().primaryKey(),
  userId: int("userId").notNull(),
  totalRiskScore: int("totalRiskScore").notNull(),
  riskLevel: varchar("riskLevel", { length: 50 }).notNull(),
  totalDebt: int("totalDebt").notNull(),
  monthlyIncome: int("monthlyIncome").default(0),
  monthlyExpenses: int("monthlyExpenses").default(0),
  availableForDebt: int("availableForDebt").default(0),
  creditorCount: int("creditorCount").default(0),
  hasEnforcement: boolean("hasEnforcement").default(false),
  hasWarningLetters: boolean("hasWarningLetters").default(false),
  debtsData: text("debtsData"),
  actionsData: text("actionsData"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow(),
});

export type Diagnosis = typeof diagnoses.$inferSelect;
export type InsertDiagnosis = typeof diagnoses.$inferInsert;
```

4. SERVER CODE

server/routers.ts

```
import { COOKIE_NAME } from "@shared/const";
import { getSessionCookieOptions } from "./_core/cookies";
import { systemRouter } from "./_core/systemRouter";
import { publicProcedure, router } from "./_core/trpc";
import { casesRouter } from "./routers/cases";
import { documentsRouter } from "./routers/documents";
import { tasksRouter } from "./routers/tasks";
import { consentRouter } from "./routers/consent";
import { diagnosisRouter } from "./routers/diagnosis";

export const appRouter = router({
  system: systemRouter,
  auth: router({
    me: publicProcedure.query(opts => opts.ctx.user),
    logout: publicProcedure.mutation(({ ctx }) => {
      const cookieOptions = getSessionCookieOptions(ctx.req);
      ctx.res.clearCookie(COOKIE_NAME, { ...cookieOptions, maxAge: -1 });
      return {
        success: true,
      } as const;
    }),
  }),

  // Freedom routers
  cases: casesRouter,
  documents: documentsRouter,
  tasks: tasksRouter,
  consent: consentRouter,
  diagnosis: diagnosisRouter,
});

export type AppRouter = typeof appRouter;
```

server/db.ts

```ts
import { eq } from "drizzle-orm";
import { drizzle } from "drizzle-orm/mysql2";
import { InsertUser, users } from "../drizzle/schema";
import { ENV } from './_core/env';

let _db: ReturnType<typeof drizzle> | null = null;

// Lazily create the drizzle instance so local tooling can run without a DB.
export async function getDb() {
  if (!_db && process.env.DATABASE_URL) {
    try {
      _db = drizzle(process.env.DATABASE_URL);
    } catch (error) {
      console.warn("[Database] Failed to connect:", error);
      _db = null;
    }
  }
  return _db;
}

export async function upsertUser(user: InsertUser): Promise<void> {
  if (!user.openId) {
    throw new Error("User openId is required for upsert");
  }

  const db = await getDb();
  if (!db) {
    console.warn("[Database] Cannot upsert user: database not available");
    return;
  }

  try {
    const values: InsertUser = {
      openId: user.openId,
    };
    const updateSet: Record<string, unknown> = {};

    const textFields = ["name", "email", "loginMethod"] as const;
    type TextField = (typeof textFields)[number];

    const assignNullable = (field: TextField) => {
      const value = user[field];
      if (value === undefined) return;
```

```
      const normalized = value ?? null;
      values[field] = normalized;
      updateSet[field] = normalized;
    };

    textFields.forEach(assignNullable);

    if (user.lastSignedIn !== undefined) {
      values.lastSignedIn = user.lastSignedIn;
      updateSet.lastSignedIn = user.lastSignedIn;
    }
    if (user.role !== undefined) {
      values.role = user.role;
      updateSet.role = user.role;
    } else if (user.openId === ENV.ownerOpenId) {
      values.role = 'admin';
      updateSet.role = 'admin';
    }

    if (!values.lastSignedIn) {
      values.lastSignedIn = new Date();
    }

    if (Object.keys(updateSet).length === 0) {
      updateSet.lastSignedIn = new Date();
    }

    await db.insert(users).values(values).onDuplicateKeyUpdate({
      set: updateSet,
    });
  } catch (error) {
    console.error("[Database] Failed to upsert user:", error);
    throw error;
  }
}

export async function getUserByOpenId(openId: string) {
  const db = await getDb();
  if (!db) {
    console.warn("[Database] Cannot get user: database not available");
    return undefined;
  }

  const result = await db.select().from(users).where(eq(users.openId,
openId)).limit(1);
```

```
  return result.length > 0 ? result[0] : undefined;
}

// TODO: add feature queries here as your schema grows.
```

server/queries.ts

```ts
import { getDb } from './db';
import { eq, and, desc, sql } from 'drizzle-orm';
import {
  debtProfiles,
  cases,
  documents,
  tasks,
  consentRecords,
  professionalProfiles,
  notifications,
} from '../drizzle/schema';
import type {
  InsertDebtProfile,
  InsertCase,
  InsertDocument,
  InsertTask,
  InsertConsentRecord,
  InsertNotification,
} from '../drizzle/schema';

/**
 * Create a new debt profile for a debtor
 */
export async function createDebtProfile(
  userId: number,
  data: Omit<InsertDebtProfile, 'userId'>
): Promise<any> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  const result = await db.insert(debtProfiles).values({
    userId,
    ...data,
  });

  // Drizzle MySQL returns { insertId: number }
  return { insertId: (result as any).insertId };
}

/**
 * Get debt profile by user ID
 */
export async function getDebtProfileByUserId(userId: number): Promise<any> {
```

```
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  const result = await db
    .select()
    .from(debtProfiles)
    .where(eq(debtProfiles.userId, userId))
    .limit(1);

  return result[0] || null;
}

/**
 * Update debt profile
 */
export async function updateDebtProfile(
  debtProfileId: number,
  data: Partial<InsertDebtProfile>
): Promise<void> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  await db
    .update(debtProfiles)
    .set(data)
    .where(eq(debtProfiles.id, debtProfileId));
}

/**
 * Create a new case (link debtor to professional)
 */
export async function createCase(data: InsertCase): Promise<any> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  const result = await db.insert(cases).values(data);
  return result;
}

/**
 * Get case by ID
 */
export async function getCaseById(caseId: number): Promise<any> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');
```

```
  const result = await db
    .select()
    .from(cases)
    .where(eq(cases.id, caseId))
    .limit(1);

  return result[0] || null;
}

/**
 * Get cases by debt profile ID (with debt profile details)
 */
export async function getCasesByDebtProfileId(debtProfileId: number):
Promise<any[]> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  return await db
    .select({
      id: cases.id,
      debtProfileId: cases.debtProfileId,
      professionalUserId: cases.professionalUserId,
      status: cases.status,
      matchingScore: cases.matchingScore,
      createdAt: cases.createdAt,
      updatedAt: cases.updatedAt,
      totalDebtAmount: debtProfiles.totalDebtAmount,
      debtType: debtProfiles.debtType,
      severity: debtProfiles.severity,
      persona: debtProfiles.persona,
    })
    .from(cases)
    .innerJoin(debtProfiles, eq(cases.debtProfileId, debtProfiles.id))
    .where(eq(cases.debtProfileId, debtProfileId));
}

/**
 * Get cases assigned to a professional (with debt profile details)
 */
export async function getCasesByProfessionalId(professionalUserId: number):
Promise<any[]> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  return await db
    .select({
```

```
      id: cases.id,
      debtProfileId: cases.debtProfileId,
      professionalUserId: cases.professionalUserId,
      status: cases.status,
      matchingScore: cases.matchingScore,
      createdAt: cases.createdAt,
      updatedAt: cases.updatedAt,
      totalDebtAmount: debtProfiles.totalDebtAmount,
      debtType: debtProfiles.debtType,
      severity: debtProfiles.severity,
      persona: debtProfiles.persona,
    })
    .from(cases)
    .innerJoin(debtProfiles, eq(cases.debtProfileId, debtProfiles.id))
    .where(eq(cases.professionalUserId, professionalUserId));
}

/**
 * Update case status
 */
export async function updateCaseStatus(
  caseId: number,
  status: 'open' | 'in_progress' | 'closed' | 'on_hold'
): Promise<void> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  await db
    .update(cases)
    .set({ status, updatedAt: new Date() })
    .where(eq(cases.id, caseId));
}

/**
 * Upload document
 */
export async function uploadDocument(data: InsertDocument): Promise<any> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  const result = await db.insert(documents).values(data);
  return result;
}

/**
 * Get documents by case ID
```

```
 */
export async function getDocumentsByCaseId(caseId: number): Promise<any[]> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  return await db
    .select()
    .from(documents)
    .where(eq(documents.caseId, caseId))
    .orderBy(desc(documents.createdAt));
}

/**
 * Get document by ID
 */
export async function getDocumentById(documentId: number): Promise<any> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  const result = await db
    .select()
    .from(documents)
    .where(eq(documents.id, documentId))
    .limit(1);

  return result[0] || null;
}

/**
 * Update document (mark as OCR processed, add summary, etc.)
 */
export async function updateDocument(
  documentId: number,
  data: Partial<InsertDocument>
): Promise<void> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  await db
    .update(documents)
    .set({ ...data, updatedAt: new Date() })
    .where(eq(documents.id, documentId));
}

/**
 * Create task
```

```
 */
export async function createTask(data: InsertTask): Promise<any> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  const result = await db.insert(tasks).values(data);
  return result;
}

/**
 * Get tasks by case ID
 */
export async function getTasksByCaseId(caseId: number): Promise<any[]> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  return await db
    .select()
    .from(tasks)
    .where(eq(tasks.caseId, caseId))
    .orderBy(desc(tasks.dueDate));
}

/**
 * Get tasks assigned to user
 */
export async function getTasksByAssignedUser(userId: number): Promise<any[]>
{
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  return await db
    .select()
    .from(tasks)
    .where(eq(tasks.assignedToUserId, userId))
    .orderBy(desc(tasks.dueDate));
}

/**
 * Update task status
 */
export async function updateTaskStatus(
  taskId: number,
  status: 'pending' | 'in_progress' | 'completed' | 'blocked'
): Promise<void> {
  const db = await getDb();
```

```
  if (!db) throw new Error('Database not available');

  const completedAt = status === 'completed' ? new Date() : null;

  await db
    .update(tasks)
    .set({ status, completedAt, updatedAt: new Date() })
    .where(eq(tasks.id, taskId));
}

/**
 * Record consent
 */
export async function recordConsent(data: InsertConsentRecord): Promise<any>
{
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  const result = await db.insert(consentRecords).values(data);
  return result;
}

/**
 * Get user consent records
 */
export async function getUserConsentRecords(userId: number): Promise<any[]>
{
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  return await db
    .select()
    .from(consentRecords)
    .where(eq(consentRecords.userId, userId))
    .orderBy(desc(consentRecords.createdAt));
}

/**
 * Check if user has given specific consent
 */
export async function hasUserConsent(
  userId: number,
  consentType: 'data_processing' | 'ai_analysis' | 'professional_sharing' |
'communication'
): Promise<boolean> {
  const db = await getDb();
```

```
  if (!db) throw new Error('Database not available');

  const result = await db
    .select()
    .from(consentRecords)
    .where(
      and(
        eq(consentRecords.userId, userId),
        eq(consentRecords.consentType, consentType),
        eq(consentRecords.consentGiven, true)
      )
    )
    .limit(1);

  return result.length > 0;
}

/**
 * Create notification
 */
export async function createNotification(data: InsertNotification):
Promise<any> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  const result = await db.insert(notifications).values(data);
  return result;
}

/**
 * Get user notifications
 */
export async function getUserNotifications(userId: number, limit: number =
50): Promise<any[]> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  return await db
    .select()
    .from(notifications)
    .where(eq(notifications.userId, userId))
    .orderBy(desc(notifications.createdAt))
    .limit(limit);
}

/**
```

```
 * Mark notification as read
 */
export async function markNotificationAsRead(notificationId: number):
Promise<void> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  await db
    .update(notifications)
    .set({ isRead: true, readAt: new Date() })
    .where(eq(notifications.id, notificationId));
}

/**
 * Get professional profile
 */
export async function getProfessionalProfile(userId: number): Promise<any> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  const result = await db
    .select()
    .from(professionalProfiles)
    .where(eq(professionalProfiles.userId, userId))
    .limit(1);

  return result[0] || null;
}

/**
 * Create or update professional profile
 */
export async function upsertProfessionalProfile(
  userId: number,
  data: Partial<InsertDebtProfile>
): Promise<void> {
  const db = await getDb();
  if (!db) throw new Error('Database not available');

  const existing = await getProfessionalProfile(userId);

  if (existing) {
    await db
      .update(professionalProfiles)
      .set({ ...data, updatedAt: new Date() })
      .where(eq(professionalProfiles.userId, userId));
```

```
  } else {
    await db.insert(professionalProfiles).values({
      userId,
      ...data,
    });
  }
}
```

server/routers/diagnosis.ts

```
import { router, protectedProcedure } from "../_core/trpc";
import { z } from "zod";
import { getDb } from "../db";
import { diagnoses } from "../../drizzle/schema";
import { eq } from "drizzle-orm";

export const diagnosisRouter = router({
  save: protectedProcedure
    .input(
      z.object({
        riskScore: z.number(),
        riskLevel: z.string(),
        totalDebt: z.number(),
        monthlyIncome: z.number(),
        monthlyExpenses: z.number(),
        availableForDebt: z.number(),
        creditorCount: z.number(),
        hasEnforcement: z.boolean(),
        hasWarningLetters: z.boolean(),
        debts: z.array(z.any()),
        actions: z.array(z.any()),
      })
    )
    .mutation(async ({ ctx, input }) => {
      const db = await getDb();
      if (!db) throw new Error("Database not available");

      await db
        .insert(diagnoses)
        .values({
          userId: ctx.user.id,
          totalRiskScore: input.riskScore,
          riskLevel: input.riskLevel,
          totalDebt: input.totalDebt,
          monthlyIncome: input.monthlyIncome,
          monthlyExpenses: input.monthlyExpenses,
          availableForDebt: input.availableForDebt,
          creditorCount: input.creditorCount,
          hasEnforcement: input.hasEnforcement,
          hasWarningLetters: input.hasWarningLetters,
          debtsData: JSON.stringify(input.debts),
          actionsData: JSON.stringify(input.actions),
        });
```

```
      return { success: true };
    }),

  getMine: protectedProcedure.query(async ({ ctx }) => {
    const db = await getDb();
    if (!db) return null;

    const result = await db
      .select()
      .from(diagnoses)
      .where(eq(diagnoses.userId, ctx.user.id))
      .orderBy(diagnoses.createdAt)
      .limit(1);

    return result[0] || null;
  }),

  hasProfile: protectedProcedure.query(async ({ ctx }) => {
    const db = await getDb();
    if (!db) return false;

    const result = await db
      .select()
      .from(diagnoses)
      .where(eq(diagnoses.userId, ctx.user.id))
      .limit(1);

    return result.length > 0;
  }),
});
```

5. CLIENT CODE

client/src/App.tsx

```
import { Toaster } from "@/components/ui/sonner";
import { TooltipProvider } from "@/components/ui/tooltip";
import NotFound from "@/pages/NotFound";
import { Route, Switch } from "wouter";
import ErrorBoundary from "./components/ErrorBoundary";
import { ThemeProvider } from "./contexts/ThemeContext";
import Home from "./pages/Home";

import Dashboard from './pages/Dashboard';
import TriageWizard from './pages/TriageWizard';
import ProfessionalDiagnosis from './pages/ProfessionalDiagnosis';
import Diagnosis from './pages/Diagnosis';
import Profile from './pages/Profile';
import Letters from './pages/Letters';
import Calculator from './pages/Calculator';
import DebtTracker from './pages/DebtTracker';
import DocumentScanner from './pages/DocumentScanner';
import Lawyers from './pages/Lawyers';

function Router() {
  // make sure to consider if you need authentication for certain routes
  return (
    <Switch>
      <Route path={"/"} component={Home} />
      <Route path={"/dashboard"} component={Dashboard} />
      <Route path={"/triage"} component={TriageWizard} />
      <Route path={"/diagnosis-professional"} component=
{ProfessionalDiagnosis} />
      <Route path={"/diagnosis"} component={Diagnosis} />
      <Route path={"/profile"} component={Profile} />
      <Route path={"/letters"} component={Letters} />
      <Route path={"/calculator"} component={Calculator} />
      <Route path={"/tracker"} component={DebtTracker} />
      <Route path={"/scanner"} component={DocumentScanner} />
      <Route path={"/lawyers"} component={Lawyers} />
      <Route path={"/404"} component={NotFound} />
      {/* Final fallback route */}
      <Route component={NotFound} />
    </Switch>
  );
```

```
}

// NOTE: About Theme
// - First choose a default theme according to your design style (dark or
light bg), than change color palette in index.css
//   to keep consistent foreground/background color across components
// - If you want to make theme switchable, pass `switchable` ThemeProvider
and use `useTheme` hook

function App() {
  return (
    <ErrorBoundary>
      <ThemeProvider
        defaultTheme="light"
        // switchable
      >
        <TooltipProvider>
          <Toaster />
          <Router />
        </TooltipProvider>
      </ThemeProvider>
    </ErrorBoundary>
  );
}

export default App;
```

client/src/main.tsx

```
import { trpc } from "@/lib/trpc";
import { UNAUTHED_ERR_MSG } from '@shared/const';
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { httpBatchLink, TRPCClientError } from "@trpc/client";
import { createRoot } from "react-dom/client";
import superjson from "superjson";
import App from "./App";
import { getLoginUrl } from "./const";
import "./index.css";

const queryClient = new QueryClient();

const redirectToLoginIfUnauthorized = (error: unknown) => {
  if (!(error instanceof TRPCClientError)) return;
  if (typeof window === "undefined") return;

  const isUnauthorized = error.message === UNAUTHED_ERR_MSG;

  if (!isUnauthorized) return;

  window.location.href = getLoginUrl();
};

queryClient.getQueryCache().subscribe(event => {
  if (event.type === "updated" && event.action.type === "error") {
    const error = event.query.state.error;
    redirectToLoginIfUnauthorized(error);
    console.error("[API Query Error]", error);
  }
});

queryClient.getMutationCache().subscribe(event => {
  if (event.type === "updated" && event.action.type === "error") {
    const error = event.mutation.state.error;
    redirectToLoginIfUnauthorized(error);
    console.error("[API Mutation Error]", error);
  }
});

const trpcClient = trpc.createClient({
  links: [
    httpBatchLink({
      url: "/api/trpc",
```

```
      transformer: superjson,
      fetch(input, init) {
        return globalThis.fetch(input, {
          ...(init ?? {}),
          credentials: "include",
        });
      },
    }),
  ],
});

createRoot(document.getElementById("root")!).render(
  <trpc.Provider client={trpcClient} queryClient={queryClient}>
    <QueryClientProvider client={queryClient}>
      <App />
    </QueryClientProvider>
  </trpc.Provider>
);
```

client/src/lib/trpc.ts

```
import { createTRPCReact } from "@trpc/react-query";
import type { AppRouter } from "../../../server/routers";

export const trpc = createTRPCReact<AppRouter>();
```

6. CLIENT PAGES

Calculator.tsx

```tsx
import DashboardLayout from '@/components/DashboardLayout';
import { Card, CardContent, CardHeader, CardTitle } from
'@/components/ui/card';

export default function Calculator() {
  return (
    <DashboardLayout>
      <div className="space-y-8">
        <div>
          <h1 className="text-3xl font-bold text-white mb-2">מחשבון חוב</h1>
          <p className="text-slate-400">חשב את התשלום החודשי שלך</p>
        </div>

        <Card className="bg-slate-800 border-slate-700">
          <CardHeader>
            <CardTitle className="text-white">מחשבון חוב</CardTitle>
          </CardHeader>
          <CardContent>
            <p className="text-slate-300">דף מחשבון חוב - בבנייה</p>
          </CardContent>
        </Card>
      </div>
    </DashboardLayout>
  );
}
```

Dashboard.tsx

```
import { useAuth } from '@/_core/hooks/useAuth';
import DashboardLayout from '@/components/DashboardLayout';
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
'@/components/ui/card';
import { Button } from '@/components/ui/button';
import { Plus, FileText, CheckSquare, Users, TrendingDown, AlertCircle,
ChevronRight, Mail, Calculator, BarChart3, Scan, Scale } from 'lucide-
react';
import { trpc } from '@/lib/trpc';
import { useLocation } from 'wouter';
import { useState } from 'react';

export default function Dashboard() {
  const { user } = useAuth();
  const [, setLocation] = useLocation();
  const [selectedCaseId, setSelectedCaseId] = useState<number | null>(null);
  const { data: cases, isLoading } = trpc.cases.getMyCases.useQuery();

  if (isLoading) {
    return (
      <DashboardLayout>
        <div className="text-center py-12">טוען...</div>
      </DashboardLayout>
    );
  }

  const currentCase = selectedCaseId
    ? cases?.find(c => c.id === selectedCaseId)
    : cases?.[0];

  if (!currentCase && cases?.length === 0) {
    return (
      <DashboardLayout>
        <div className="space-y-8">
          <div>
            <h1 className="text-3xl font-bold text-white mb-2">ברוכים הבאים,
{user?.name}</h1>
            <p className="text-slate-400">בואו נתחיל בסיווג החוב שלך</p>
          </div>

          <Card className="bg-slate-800 border-slate-700">
            <CardContent className="pt-8 pb-8">
              <div className="text-center">
```

```
                <AlertCircle className="w-12 h-12 text-slate-400 mx-auto mb-
4" />
                <h3 className="text-lg font-semibold text-white mb-2">עדיין
לא יצרת תיק</h3>
                <p className="text-slate-400 mb-6">
                  בואו נתחיל בסיווג החוב שלך כדי לקבל תוכנית פעולה מותאמת אישית
                </p>
                <Button
                  onClick={() => setLocation('/diagnosis-professional')}
                  className="bg-blue-600 hover:bg-blue-700"
                >
                  <Plus className="w-4 h-4 ml-2" />
                  התחל אבחון מקצועי
                </Button>
              </div>
            </CardContent>
          </Card>
        </div>
      </DashboardLayout>
    );
  }

  return (
    <DashboardLayout>
      <div className="space-y-8">
        {/* Header */}
        <div>
          <h1 className="text-3xl font-bold text-white mb-2">ברוכים הבאים,
{user?.name} 👋</h1>
          <p className="text-slate-400">הנה סיכום המצב של החובות שלך וצעדיך
הבאים</p>
        </div>

        {/* Case Selector */}
        {cases && cases.length > 0 && (
          <div className="flex items-center gap-3 flex-wrap">
            <span className="text-sm font-medium text-slate-300">בחר תיק:
</span>
            <div className="flex gap-2 flex-wrap">
              {cases.map((caseItem) => (
                <button
                  key={caseItem.id}
                  onClick={() => setSelectedCaseId(caseItem.id)}
                  className={`px-3 py-1.5 rounded-lg text-sm transition-all
${
                    selectedCaseId === caseItem.id || (!selectedCaseId &&
```

```
caseItem.id === cases[0]?.id)
                    ? 'bg-blue-600 text-white'
                    : 'bg-slate-700 text-slate-300 hover:bg-slate-600'
                }`}
                >
                  {caseItem.debtType.replace('_', ' ')} - ₪
{caseItem.totalDebtAmount}
                </button>
              ))}
            </div>
            <Button
              size="sm"
              onClick={() => setLocation('/diagnosis-professional')}
              className="ml-auto bg-green-600 hover:bg-green-700"
            >
              <Plus className="w-4 h-4 ml-2" />
              הוסף חוב חדש
            </Button>
          </div>
        )}

        {/* 6 Main Cards Grid */}
        {currentCase && (
          <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3
gap-6">
            {/* Card 1: Debt Status */}
            <Card className="bg-slate-800 border-slate-700 hover:border-
slate-600 transition-colors">
              <CardHeader className="pb-3">
                <CardTitle className="text-white flex items-center gap-2
text-lg">
                  <TrendingDown className="w-5 h-5 text-red-400" />
                  מצב החוב שלי
                </CardTitle>
              </CardHeader>
              <CardContent className="space-y-4">
                <div className="bg-slate-700/50 rounded-lg p-4">
                  <p className="text-sm text-slate-400 mb-1">סכום כולל</p>
                  <p className="text-3xl font-bold text-white">
                    ₪{currentCase.totalDebtAmount.toLocaleString('he-IL')}
                  </p>
                </div>
                <div className="grid grid-cols-2 gap-3">
                  <div>
                    <p className="text-xs text-slate-400">חומרה</p>
                    <div className="flex items-center gap-2 mt-1">
```

```
                    <div className={`w-3 h-3 rounded-full ${
                      currentCase.severity === 'low' ? 'bg-green-500' :
                      currentCase.severity === 'medium' ? 'bg-yellow-500'
:
                      currentCase.severity === 'high' ? 'bg-orange-500' :
                      'bg-red-500'
                    }`} />
                    <span className="text-sm text-white font-medium">
                      {currentCase.severity === 'low' && 'נמוכה'}
                      {currentCase.severity === 'medium' && 'בינונית'}
                      {currentCase.severity === 'high' && 'גבוהה'}
                      {currentCase.severity === 'critical' && 'קריטית'}
                    </span>
                  </div>
                </div>
                <div>
                  <p className="text-xs text-slate-400">סוג חוב</p>
                  <p className="text-sm text-white font-medium mt-1
capitalize">
                    {currentCase.debtType.replace('_', ' ')}
                  </p>
                </div>
              </div>
            </CardContent>
          </Card>

          {/* Card 2: Letters Generator */}
          <Card className="bg-slate-800 border-slate-700 hover:border-
slate-600 transition-colors cursor-pointer" onClick={() =>
setLocation('/letters')}>
            <CardHeader className="pb-3">
              <CardTitle className="text-white flex items-center gap-2
text-lg">
                <Mail className="w-5 h-5 text-blue-400" />
                מחולל מכתבים
              </CardTitle>
            </CardHeader>
            <CardContent className="space-y-3">
              <div className="bg-slate-700/50 rounded-lg p-4 text-center">
                <p className="text-sm text-slate-300">צור מכתבי דרישה
ותשובות</p>
                <p className="text-xs text-slate-500 mt-1">לנושים ובנקים</p>
              </div>
              <Button
                className="w-full bg-blue-600 hover:bg-blue-700 text-
white"
```

```
                onClick={() => setLocation('/letters')}
              >
                פתח מחולל <ChevronRight className="w-4 h-4 ml-2" />
              </Button>
            </CardContent>
          </Card>

          {/* Card 3: Debt Calculator */}
          <Card className="bg-slate-800 border-slate-700 hover:border-
slate-600 transition-colors cursor-pointer" onClick={() =>
setLocation('/calculator')}>
            <CardHeader className="pb-3">
              <CardTitle className="text-white flex items-center gap-2
text-lg">
                <Calculator className="w-5 h-5 text-green-400" />
                מחשבון חוב
              </CardTitle>
            </CardHeader>
            <CardContent className="space-y-3">
              <div className="bg-slate-700/50 rounded-lg p-4 text-center">
                <p className="text-sm text-slate-300">חשב התחייבויות
חודשיות</p>
                <p className="text-xs text-slate-500 mt-1">וריביות</p>
              </div>
              <Button
                className="w-full bg-green-600 hover:bg-green-700 text-
white"
                onClick={() => setLocation('/calculator')}
              >
                חשב עכשיו <ChevronRight className="w-4 h-4 ml-2" />
              </Button>
            </CardContent>
          </Card>

          {/* Card 4: Debt Tracker */}
          <Card className="bg-slate-800 border-slate-700 hover:border-
slate-600 transition-colors cursor-pointer" onClick={() =>
setLocation('/tracker')}>
            <CardHeader className="pb-3">
              <CardTitle className="text-white flex items-center gap-2
text-lg">
                <BarChart3 className="w-5 h-5 text-purple-400" />
                עקבות חוב
              </CardTitle>
            </CardHeader>
            <CardContent className="space-y-3">
```

```
                <div className="bg-slate-700/50 rounded-lg p-4 text-center">
                  <p className="text-sm text-slate-300">עקוב אחר התקדמות</p>
                  <p className="text-xs text-slate-500 mt-1">והשלם חובות</p>
                </div>
                <Button
                  className="w-full bg-purple-600 hover:bg-purple-700 text-
white"
                  onClick={() => setLocation('/tracker')}
                >
                  עקוב <ChevronRight className="w-4 h-4 ml-2" />
                </Button>
              </CardContent>
            </Card>

            {/* Card 5: Document Scanner */}
            <Card className="bg-slate-800 border-slate-700 hover:border-
slate-600 transition-colors cursor-pointer" onClick={() =>
setLocation('/scanner')}>
              <CardHeader className="pb-3">
                <CardTitle className="text-white flex items-center gap-2
text-lg">
                  <Scan className="w-5 h-5 text-orange-400" />
                  סריקת מסמכים
                </CardTitle>
              </CardHeader>
              <CardContent className="space-y-3">
                <div className="bg-slate-700/50 rounded-lg p-4 text-center">
                  <p className="text-sm text-slate-300">סרוק מסמכים עם
OCR</p>
                  <p className="text-xs text-slate-500 mt-1">חילוץ נתונים
אוטומטי</p>
                </div>
                <Button
                  className="w-full bg-orange-600 hover:bg-orange-700 text-
white"
                  onClick={() => setLocation('/scanner')}
                >
                  סרוק <ChevronRight className="w-4 h-4 ml-2" />
                </Button>
              </CardContent>
            </Card>

            {/* Card 6: Lawyers & Advisors */}
            <Card className="bg-slate-800 border-slate-700 hover:border-
slate-600 transition-colors cursor-pointer" onClick={() =>
setLocation('/lawyers')}>
```

```
              <CardHeader className="pb-3">
                <CardTitle className="text-white flex items-center gap-2
text-lg">
                  <Scale className="w-5 h-5 text-indigo-400" />
                  עורכי דין
                </CardTitle>
              </CardHeader>
              <CardContent className="space-y-3">
                <div className="bg-slate-700/50 rounded-lg p-4 text-center">
                  <p className="text-sm text-slate-300">מצא עורך דין או
יועץ</p>
                  <p className="text-xs text-slate-500 mt-1">מומחה בחובות</p>
                </div>
                <Button
                  className="w-full bg-indigo-600 hover:bg-indigo-700 text-
white"
                  onClick={() => setLocation('/lawyers')}
                >
                  חפש <ChevronRight className="w-4 h-4 ml-2" />
                </Button>
              </CardContent>
            </Card>
          </div>
        )}
      </div>
    </DashboardLayout>
  );
}
```

DebtTracker.tsx

```tsx
import DashboardLayout from '@/components/DashboardLayout';
import { Card, CardContent, CardHeader, CardTitle } from
'@/components/ui/card';

export default function DebtTracker() {
  return (
    <DashboardLayout>
      <div className="space-y-8">
        <div>
          <h1 className="text-3xl font-bold text-white mb-2">עקבות חוב</h1>
          <p className="text-slate-400">עקוב אחר ההתקדמות שלך</p>
        </div>

        <Card className="bg-slate-800 border-slate-700">
          <CardHeader>
            <CardTitle className="text-white">עקבות חוב</CardTitle>
          </CardHeader>
          <CardContent>
            <p className="text-slate-300">דף עקבות חוב - בבנייה</p>
          </CardContent>
        </Card>
      </div>
    </DashboardLayout>
  );
}
```

Diagnosis.tsx

```tsx
import DashboardLayout from '@/components/DashboardLayout';
import { Card, CardContent, CardHeader, CardTitle } from
'@/components/ui/card';

export default function Diagnosis() {
  return (
    <DashboardLayout>
      <div className="space-y-8">
        <div>
          <h1 className="text-3xl font-bold text-white mb-2">אבחון מהיר</h1>
          <p className="text-slate-400">בואו נתחיל בסיווג החוב שלך</p>
        </div>

        <Card className="bg-slate-800 border-slate-700">
          <CardHeader>
            <CardTitle className="text-white">אבחון מהיר</CardTitle>
          </CardHeader>
          <CardContent>
            <p className="text-slate-300">דף אבחון מהיר - בבנייה</p>
          </CardContent>
        </Card>
      </div>
    </DashboardLayout>
  );
}
```

DocumentScanner.tsx

```
import DashboardLayout from '@/components/DashboardLayout';
import { Card, CardContent, CardHeader, CardTitle } from
'@/components/ui/card';

export default function DocumentScanner() {
  return (
    <DashboardLayout>
      <div className="space-y-8">
        <div>
          <h1 className="text-3xl font-bold text-white mb-2">סריקת
מסמכים</h1>
          <p className="text-slate-400">העלה וסרוק מסמכים עם OCR</p>
        </div>

        <Card className="bg-slate-800 border-slate-700">
          <CardHeader>
            <CardTitle className="text-white">סריקת מסמכים</CardTitle>
          </CardHeader>
          <CardContent>
            <p className="text-slate-300">דף סריקת מסמכים - בבנייה</p>
          </CardContent>
        </Card>
      </div>
    </DashboardLayout>
  );
}
```

Home.tsx

```
import { useAuth } from '@/_core/hooks/useAuth';
import { Button } from '@/components/ui/button';
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
'@/components/ui/card';
import { CheckCircle2, Shield, Zap, Users } from 'lucide-react';
import { getLoginUrl } from '@/const';
import { useLocation } from 'wouter';

export default function Home() {
  const { user, isAuthenticated } = useAuth();
  const [, setLocation] = useLocation();

  return (
    <div className="min-h-screen bg-gradient-to-br from-slate-900 via-slate-
800 to-slate-900">
      {/* Navigation */}
      <nav className="border-b border-slate-700 bg-slate-900/50 backdrop-
blur">
        <div className="container mx-auto px-4 py-4 flex justify-between
items-center">
          <div className="text-2xl font-bold text-white ltr" dir="ltr">🔓
Freedom</div>
          {isAuthenticated && user ? (
            <Button asChild>
              <a href="/dashboard">לדשבורד שלי</a>
            </Button>
          ) : (
            <Button asChild>
              <a href={getLoginUrl()}>כניסה עם גוגל</a>
            </Button>
          )}
        </div>
      </nav>

      {/* Hero Section */}
      <section className="container mx-auto px-4 py-20">
        <div className="max-w-3xl mx-auto text-center">
          <h1 className="text-5xl font-bold text-white mb-6">
            מבינים את החוב, מסדרים את הכיוון, ומחזירים לך שליטה
          </h1>
          <p className="text-xl text-slate-300 mb-8">
            Freedom היא פלטפורמה שמסייעת לאנשים בחוב להבין איפה הם עומדים, מה
הצעד הבא, ואיך להגיע לעזרה הנכונה – בצורה ברורה, מהירה ובלי בלבול.
```

```
          </p>
          <div className="flex gap-4 justify-center">
            {isAuthenticated && user ? (
              <>
                <Button asChild size="lg" className="bg-blue-600 hover:bg-
blue-700">
                  <a href="/diagnosis-professional">התחל אבחון חדש</a>
                </Button>
                <Button asChild variant="outline" size="lg"
className="border-slate-600 text-white hover:bg-slate-800">
                  <a href="/dashboard">לדשבורד שלי</a>
                </Button>
              </>
            ) : (
              <>
                <Button asChild size="lg" className="bg-blue-600 hover:bg-
blue-700">
                  <a href={getLoginUrl()}>התחל אבחון</a>
                </Button>
                <Button asChild variant="outline" size="lg"
className="border-slate-600 text-white hover:bg-slate-800">
                  <a href="#features">למד עוד</a>
                </Button>
              </>
            )}
          </div>
        </div>
      </section>

      {/* Value Propositions */}
      <section id="features" className="container mx-auto px-4 py-20">
        <h2 className="text-3xl font-bold text-white text-center mb-12">למה
Freedom?</h2>
        <div className="grid md:grid-cols-3 gap-8">
          <Card className="bg-slate-800 border-slate-700">
            <CardHeader>
              <Zap className="w-8 h-8 text-yellow-400 mb-2" />
              <CardTitle className="text-white">אבחון מהיר</CardTitle>
            </CardHeader>
            <CardContent className="text-slate-300">
              להבין את מצב החוב שלך בלי להסתבך. תוך 3 דקות תדע בדיוק איפה אתה
עומד.
            </CardContent>
          </Card>

          <Card className="bg-slate-800 border-slate-700">
```

```
              <CardHeader>
                <Users className="w-8 h-8 text-blue-400 mb-2" />
                <CardTitle className="text-white">הכוונה מדויקת</CardTitle>
              </CardHeader>
              <CardContent className="text-slate-300">
                לדעת מה לעשות עכשיו ולאן לפנות. אנחנו מחברים אותך למומחה הנכון בדיוק
בזמן הנכון.
              </CardContent>
            </Card>

            <Card className="bg-slate-800 border-slate-700">
              <CardHeader>
                <Shield className="w-8 h-8 text-green-400 mb-2" />
                <CardTitle className="text-white">סדר וביטחון</CardTitle>
              </CardHeader>
              <CardContent className="text-slate-300">
                להפחית לחץ ולהחזיר תחושת שליטה. כל המסמכים שלך מוצפנים וממשק אחד
לכל הדברים.
              </CardContent>
            </Card>
          </div>
        </section>

        {/* How It Works */}
        <section className="container mx-auto px-4 py-20 bg-slate-800/50
rounded-lg">
          <h2 className="text-3xl font-bold text-white text-center mb-12">איך
זה עובד?</h2>
          <div className="grid md:grid-cols-4 gap-8">
            {[
              {
                number: '1',
                title: 'אבחון',
                description: 'תענה על כמה שאלות פשוטות על החוב שלך',
              },
              {
                number: '2',
                title: 'סיווג',
                description: 'המערכת תסווג את מצבך ותציע מומחה מתאים',
              },
              {
                number: '3',
                title: 'חיבור',
                description: 'תתחבר למומחה בלחיצת כפתור אחת',
              },
              {
```

```
              number: '4',
              title: 'ליווי',
              description: 'תקבל תזכורות, משימות ועדכונים שוטפים',
            },
          ].map((step) => (
            <div key={step.number} className="text-center">
              <div className="w-12 h-12 bg-blue-600 rounded-full flex items-
center justify-center mx-auto mb-4">
                <span className="text-white font-bold text-lg">{step.number}
</span>
              </div>
              <h3 className="text-lg font-semibold text-white mb-2">
{step.title}</h3>
              <p className="text-slate-300 text-sm">{step.description}</p>
            </div>
          ))}
        </div>
      </section>

      {/* CTA Section */}
      <section className="container mx-auto px-4 py-20 text-center">
        <h2 className="text-3xl font-bold text-white mb-6">מוכן להתחיל?</h2>
        <p className="text-xl text-slate-300 mb-8">
          בדוק איפה אתה עומד עכשיו. זה חינם, מהיר ובלי התחייבות.
        </p>
        <Button asChild size="lg" className="bg-green-600 hover:bg-green-
700">
          <a href={getLoginUrl()}>בוא נתחיל</a>
        </Button>
      </section>

      {/* Footer */}
      <footer className="border-t border-slate-700 bg-slate-900/50 py-8">
        <div className="container mx-auto px-4 text-center text-slate-400
text-sm">
          <p>© 2026 Freedom - כל הזכויות שמורות</p>
          <p className="mt-2">Freedom (2024) 13 תיקון ,מתאימה לחוק הגנת הפרטיות
</p>
        </div>
      </footer>
    </div>
  );
}
```

Lawyers.tsx

```
import DashboardLayout from '@/components/DashboardLayout';
import { Card, CardContent, CardHeader, CardTitle } from
'@/components/ui/card';

export default function Lawyers() {
  return (
    <DashboardLayout>
      <div className="space-y-8">
        <div>
          <h1 className="text-3xl font-bold text-white mb-2">עורכי דין
ויועצים</h1>
          <p className="text-slate-400">מצא את המומחה הנכון לך</p>
        </div>

        <Card className="bg-slate-800 border-slate-700">
          <CardHeader>
            <CardTitle className="text-white">עורכי דין ויועצים</CardTitle>
          </CardHeader>
          <CardContent>
            <p className="text-slate-300">דף עורכי דין ויועצים - בבנייה</p>
          </CardContent>
        </Card>
      </div>
    </DashboardLayout>
  );
}
```

Letters.tsx

```
import DashboardLayout from '@/components/DashboardLayout';
import { Card, CardContent, CardHeader, CardTitle } from
'@/components/ui/card';

export default function Letters() {
  return (
    <DashboardLayout>
      <div className="space-y-8">
        <div>
          <h1 className="text-3xl font-bold text-white mb-2">מחולל
מכתבים</h1>
          <p className="text-slate-400">צור מכתבי דרישה ותביעה</p>
        </div>

        <Card className="bg-slate-800 border-slate-700">
          <CardHeader>
            <CardTitle className="text-white">מחולל מכתבים</CardTitle>
          </CardHeader>
          <CardContent>
            <p className="text-slate-300">דף מחולל מכתבים - בבנייה</p>
          </CardContent>
        </Card>
      </div>
    </DashboardLayout>
  );
}
```

NotFound.tsx

```
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import { AlertCircle, Home } from "lucide-react";
import { useLocation } from "wouter";

export default function NotFound() {
  const [, setLocation] = useLocation();

  const handleGoHome = () => {
    setLocation("/");
  };

  return (
    <div className="min-h-screen w-full flex items-center justify-center bg-
gradient-to-br from-slate-50 to-slate-100">
      <Card className="w-full max-w-lg mx-4 shadow-lg border-0 bg-white/80
backdrop-blur-sm">
        <CardContent className="pt-8 pb-8 text-center">
          <div className="flex justify-center mb-6">
            <div className="relative">
              <div className="absolute inset-0 bg-red-100 rounded-full
animate-pulse" />
              <AlertCircle className="relative h-16 w-16 text-red-500" />
            </div>
          </div>

          <h1 className="text-4xl font-bold text-slate-900 mb-2">404</h1>

          <h2 className="text-xl font-semibold text-slate-700 mb-4">
            Page Not Found
          </h2>

          <p className="text-slate-600 mb-8 leading-relaxed">
            Sorry, the page you are looking for doesn't exist.
            <br />
            It may have been moved or deleted.
          </p>

          <div
            id="not-found-button-group"
            className="flex flex-col sm:flex-row gap-3 justify-center"
          >
            <Button
```

```
              onClick={handleGoHome}
              className="bg-blue-600 hover:bg-blue-700 text-white px-6 py-
2.5 rounded-lg transition-all duration-200 shadow-md hover:shadow-lg"
            >
              <Home className="w-4 h-4 mr-2" />
              Go Home
            </Button>
          </div>
        </CardContent>
      </Card>
    </div>
  );
}
```

ProfessionalDiagnosis.tsx

```
import { useState } from 'react';
import { useAuth } from '@/_core/hooks/useAuth';
import DashboardLayout from '@/components/DashboardLayout';
import { Card, CardContent, CardHeader, CardTitle } from
'@/components/ui/card';
import { Button } from '@/components/ui/button';
import { trpc } from '@/lib/trpc';
import { toast } from 'sonner';
import { ArrowRight } from 'lucide-react';

interface FormData {
  debts: Array<{ category: string; amount: number; riskScore: number }>;
  totalRisk: number;
  totalAmount: number;
  diagnosisData: Record<string, any>;
}

export default function ProfessionalDiagnosis() {
  const { user } = useAuth();
  const [step, setStep] = useState(1);
  const [formData, setFormData] = useState<FormData>({
    debts: [],
    totalRisk: 0,
    totalAmount: 0,
    diagnosisData: {},
  });

  const saveMutation = trpc.diagnosis.save.useMutation({
    onSuccess: () => {
      toast.success('אבחון נשמר בהצלחה!');
      setStep(1);
      setFormData({
        debts: [],
        totalRisk: 0,
        totalAmount: 0,
        diagnosisData: {},
      });
    },
    onError: (error: any) => {
      toast.error(`שגיאה: ${error.message}`);
    },
  });
```

```
  const handleNext = () => {
    if (step < 4) setStep(step + 1);
  };

  const handleBack = () => {
    if (step > 1) setStep(step - 1);
  };

  const handleSubmit = async () => {
    try {
      await saveMutation.mutateAsync({
        riskScore: formData.totalRisk,
        riskLevel: formData.totalRisk > 150 ? 'critical' :
formData.totalRisk > 100 ? 'high' : formData.totalRisk > 50 ? 'medium' :
'low',
        totalDebt: formData.totalAmount,
        monthlyIncome: 0,
        monthlyExpenses: 0,
        availableForDebt: 0,
        creditorCount: formData.debts.length,
        hasEnforcement: false,
        hasWarningLetters: false,
        debts: formData.debts,
        actions: [],
      });
    } catch (error) {
      console.error('Error saving diagnosis:', error);
    }
  };

  return (
    <DashboardLayout>
      <div className="max-w-4xl mx-auto space-y-8">
        {/* Header */}
        <div>
          <h1 className="text-3xl font-bold text-white mb-2">אבחון
מקצועי</h1>
          <p className="text-slate-400">שלב {step} 4 מתוך</p>
        </div>

        {/* Progress Bar */}
        <div className="w-full bg-slate-700 rounded-full h-2">
          <div
            className="bg-blue-600 h-2 rounded-full transition-all"
            style={{ width: `${(step / 4) * 100}%` }}
          />
```

```
        </div>

        {/* Step Content */}
        <Card className="bg-slate-800 border-slate-700">
          <CardHeader>
            <CardTitle className="text-white">
              {step === 1 && 'בחר סוג חוב'}
              {step === 2 && 'הוסף פרטי חוב'}
              {step === 3 && 'שאלות אבחון'}
              {step === 4 && 'סיכום וביקורת'}
            </CardTitle>
          </CardHeader>
          <CardContent className="space-y-6">
            {/* Step 1: Category Selection */}
            {step === 1 && (
              <div className="space-y-4">
                <p className="text-slate-300">בחר את סוג החוב שלך:</p>
                <div className="grid grid-cols-1 md:grid-cols-2 gap-3">
                  {[
                    'כרטיס אשראי',
                    'הלוואה מחברת אשראי',
                    'הלוואה על מסגרת אשראי (Credit Line)',
                    'הלוואה בנקאית',
                    'הלוואה אישית',
                    'משכנתא',
                    'חובות מס',
                    'חובות עירוניים',
                    'אחר'
                  ].map((category) => (
                    <button
                      key={category}
                      onClick={() => {
                        setFormData({
                          ...formData,
                          debts: [{ category, amount: 0, riskScore: 0 }],
                        });
                        handleNext();
                      }}
                      className="p-4 bg-slate-700 hover:bg-slate-600
rounded-lg text-white text-right transition-colors"
                    >
                      {category}
                    </button>
                  ))}
                </div>
              </div>
```

```
            )}

            {/* Step 2: Debt Details */}
            {step === 2 && (
              <div className="space-y-4">
                <p className="text-slate-300">הוסף פרטי החוב:</p>
                <div className="space-y-3">
                  <input
                    type="number"
                    placeholder="סכום החוב (₪)"
                    value={formData.totalAmount}
                    onChange={(e) => setFormData({ ...formData, totalAmount:
Number(e.target.value) })}
                    className="w-full px-4 py-2 bg-slate-700 border border-
slate-600 rounded-lg text-white placeholder-slate-400"
                  />
                  <textarea
                    placeholder="הערה על החוב"
                    value={formData.diagnosisData.notes || ''}
                    onChange={(e) => setFormData({
                      ...formData,
                      diagnosisData: { ...formData.diagnosisData, notes:
e.target.value }
                    })}
                    className="w-full px-4 py-2 bg-slate-700 border border-
slate-600 rounded-lg text-white placeholder-slate-400 resize-none"
                    rows={3}
                  />
                </div>
              </div>
            )}

            {/* Step 3: Diagnosis Questions */}
            {step === 3 && (
              <div className="space-y-4">
                <p className="text-slate-300">ענה על השאלות:</p>
                <div className="space-y-3">
                  {[
                    'האם יש הוצל"פ פעיל?',
                    'האם יש הליך משפטי?',
                    'האם יש משכנתא?',
                  ].map((question, idx) => (
                    <div key={idx} className="flex items-center gap-3 p-3
bg-slate-700 rounded-lg">
                      <input
                        type="checkbox"
```

```
                      id={`q${idx}`}
                      onChange={(e) => {
                        const newData = { ...formData.diagnosisData };
                        newData[`q${idx}`] = e.target.checked;
                        setFormData({ ...formData, diagnosisData: newData
});
                      }}
                      className="w-4 h-4"
                    />
                    <label htmlFor={`q${idx}`} className="text-white
cursor-pointer flex-1">
                      {question}
                    </label>
                  </div>
                ))}
              </div>
            </div>
          )}

          {/* Step 4: Summary */}
          {step === 4 && (
            <div className="space-y-4">
              <div className="bg-slate-700 rounded-lg p-4 space-y-2">
                <p className="text-slate-400">סכום כולל:</p>
                <p className="text-2xl font-bold text-white">₪
{formData.totalAmount.toLocaleString('he-IL')}</p>
              </div>
              <div className="bg-slate-700 rounded-lg p-4 space-y-2">
                <p className="text-slate-400">רמת סיכון:</p>
                <p className="text-2xl font-bold text-white">
{formData.totalRisk}</p>
              </div>
            </div>
          )}

          {/* Navigation Buttons */}
          <div className="flex gap-3 pt-6">
            {step > 1 && (
              <Button
                onClick={handleBack}
                variant="outline"
                className="flex-1 border-slate-600 text-slate-300
hover:bg-slate-700"
              >
                <ArrowRight className="w-4 h-4 ml-2" />
                חזור
```

```
                </Button>
              )}
              {step < 4 && (
                <Button
                  onClick={handleNext}
                  className="flex-1 bg-blue-600 hover:bg-blue-700"
                >
                  הבא
                  <ArrowRight className="w-4 h-4 mr-2" />
                </Button>
              )}
              {step === 4 && (
                <Button
                  onClick={handleSubmit}
                  disabled={saveMutation.isPending}
                  className="flex-1 bg-green-600 hover:bg-green-700"
                >
                  {saveMutation.isPending ? 'שומר...' : 'סיים אבחון'}
                </Button>
              )}
            </div>
          </CardContent>
        </Card>
      </div>
    </DashboardLayout>
  );
}
```

Profile.tsx

```tsx
import DashboardLayout from '@/components/DashboardLayout';
import { Card, CardContent, CardHeader, CardTitle } from
'@/components/ui/card';

export default function Profile() {
  return (
    <DashboardLayout>
      <div className="space-y-8">
        <div>
          <h1 className="text-3xl font-bold text-white mb-2">פרופיל</h1>
          <p className="text-slate-400">פרטי החשבון שלך</p>
        </div>

        <Card className="bg-slate-800 border-slate-700">
          <CardHeader>
            <CardTitle className="text-white">פרופיל</CardTitle>
          </CardHeader>
          <CardContent>
            <p className="text-slate-300">דף פרופיל - בבנייה</p>
          </CardContent>
        </Card>
      </div>
    </DashboardLayout>
  );
}
```

TriageWizard.tsx

```tsx
import { useState } from 'react';
import { useLocation } from 'wouter';
import { Button } from '@/components/ui/button';
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
'@/components/ui/card';
import { Input } from '@/components/ui/input';
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue }
from '@/components/ui/select';
import { Textarea } from '@/components/ui/textarea';
import { toast } from 'sonner';
import { trpc } from '@/lib/trpc';
import { CheckCircle2, ChevronRight, AlertCircle } from 'lucide-react';

const DEBT_CATEGORIES = [
  { value: 'credit_card', label: 'כרטיס אשראי', icon: '💳' },
  { value: 'bank', label: 'הלוואה בנקאית', icon: '🏦' },
  { value: 'personal_loan', label: 'הלוואה אישית', icon: '👤' },
  { value: 'mortgage', label: 'משכנתא', icon: '🏠' },
  { value: 'tax', label: 'חובות מס', icon: '📋' },
  { value: 'other', label: 'אחר', icon: '📌' },
];

const SEVERITY_LEVELS = [
  { value: 'low', label: 'נמוך - בשליטה', color: 'bg-green-500' },
  { value: 'medium', label: 'בינוני - דורש תשומת לב', color: 'bg-yellow-500' },
  { value: 'high', label: 'גבוה - דחוף', color: 'bg-orange-500' },
  { value: 'critical', label: 'קריטי - סיכון מיידי', color: 'bg-red-500' },
];

interface FormData {
  totalDebtAmount: string;
  debtType: string;
  collectionActions: string;
  additionalContext: string;
}

export default function TriageWizard() {
  const [, setLocation] = useLocation();
  const [currentStep, setCurrentStep] = useState(1);
  const [formData, setFormData] = useState<FormData>({
    totalDebtAmount: '',
    debtType: '',
    collectionActions: '',
```

```
    additionalContext: '',
  });
  const [loading, setLoading] = useState(false);

  const createDebtProfile = trpc.cases.createDebtProfile.useMutation({
    onError: (error) => {
      console.error('[API Mutation Error]', error);
    },
  });

  const handleInputChange = (field: string, value: string) => {
    setFormData((prev) => ({
      ...prev,
      [field]: value,
    }));
  };

  const handleNext = () => {
    if (currentStep < 3) {
      setCurrentStep(currentStep + 1);
    }
  };

  const handleBack = () => {
    if (currentStep > 1) {
      setCurrentStep(currentStep - 1);
    }
  };

  const handleSubmit = async () => {
    setLoading(true);
    try {
      if (!formData.totalDebtAmount || !formData.debtType) {
        toast.error('אנא מלא את כל השדות הנדרשים');
        setLoading(false);
        return;
      }

      const result = await createDebtProfile.mutateAsync({
        totalDebtAmount: String(formData.totalDebtAmount),
        debtType: formData.debtType as 'bank' | 'credit_card' |
'personal_loan' | 'mortgage' | 'tax' | 'other',
        severity: 'medium' as const,
        persona: 'dana' as const,
      });
```

```
      if (!result) {
        throw new Error('לא קיבלנו תגובה מהשרת');
      }

      toast.success('הפרופיל שלך נוצר בהצלחה!');
      setLocation('/dashboard');
    } catch (error) {
      console.error('שגיאה:', error);
      toast.error('שגיאה ביצירת הפרופיל. אנא נסה שוב.');
    } finally {
      setLocation(false);
    }
  };

  const progressPercentage = (currentStep / 3) * 100;

  return (
    <div className="min-h-screen bg-gradient-to-br from-slate-900 via-slate-
800 to-slate-900 p-4">
      <div className="max-w-2xl mx-auto">
        {/* Header */}
        <div className="mb-8">
          <h1 className="text-3xl font-bold text-white mb-2">אבחון החוב
שלך</h1>
          <p className="text-slate-300">
            נשאל אותך כמה שאלות קצרות כדי להבין את מצב החוב שלך ולהתאים לך את
הצעד הבא
          </p>
        </div>

        {/* Progress Bar */}
        <div className="mb-8">
          <div className="flex items-center justify-between mb-2">
            <span className="text-sm font-medium text-slate-300">שלב
{currentStep} מתוך 3</span>
            <span className="text-sm text-slate-400">כ-2 דקות</span>
          </div>
          <div className="w-full bg-slate-700 rounded-full h-2">
            <div
              className="bg-blue-500 h-2 rounded-full transition-all
duration-300"
              style={{ width: `${progressPercentage}%` }}
            />
          </div>
        </div>
```

```
        {/* Main Card */}
        <Card className="bg-slate-800 border-slate-700 shadow-2xl">
          <CardHeader className="pb-6">
            <CardTitle className="text-white text-2xl">
              {currentStep === 1 && 'סוג החוב'}
              {currentStep === 2 && 'סכום החוב'}
              {currentStep === 3 && 'מידע נוסף'}
            </CardTitle>
            <CardDescription className="text-slate-400">
              {currentStep === 1 && 'בחר את סוג החוב הראשי שלך'}
              {currentStep === 2 && 'ספר לנו כמה אתה חייב בערך'}
              {currentStep === 3 && 'מידע נוסף שיעזור לנו להבין את המצב'}
            </CardDescription>
          </CardHeader>

          <CardContent className="space-y-6">
            {/* Step 1: Debt Type */}
            {currentStep === 1 && (
              <div className="space-y-4">
                <div className="grid grid-cols-1 md:grid-cols-2 gap-3">
                  {DEBT_CATEGORIES.map((category) => (
                    <button
                      key={category.value}
                      onClick={() => handleInputChange('debtType',
category.value)}
                      className={`p-4 rounded-lg border-2 transition-all
text-left ${
                        formData.debtType === category.value
                          ? 'border-blue-500 bg-blue-500/10'
                          : 'border-slate-600 bg-slate-700/50 hover:border-
slate-500'
                      }`}
                    >
                      <div className="text-2xl mb-2">{category.icon}</div>
                      <div className="font-medium text-white">
{category.label}</div>
                    </button>
                  ))}
                </div>
                <div className="bg-slate-700/50 border border-slate-600
rounded-lg p-3">
                  <p className="text-sm text-slate-300">
                    💡 <strong>עצה:</strong> בחר את סוג החוב הגדול ביותר או
החשוב ביותר
                  </p>
                </div>
```

```
              </div>
            )}

            {/* Step 2: Debt Amount */}
            {currentStep === 2 && (
              <div className="space-y-4">
                <div>
                  <label className="block text-sm font-medium text-white mb-
2">
                    סכום החוב הכולל (בש"ח)
                  </label>
                  <Input
                    type="number"
                    placeholder="לדוגמה: 50000"
                    value={formData.totalDebtAmount}
                    onChange={(e) => handleInputChange('totalDebtAmount',
e.target.value)}
                    className="bg-slate-700 border-slate-600 text-white
placeholder-slate-400"
                  />
                  <p className="text-xs text-slate-400 mt-2">
                    זה עוזר לנו להבין את חומרת המצב ולהתאים לך את הפתרון הטוב ביותר
                  </p>
                </div>

                {formData.totalDebtAmount && (
                  <div className="bg-slate-700/50 border border-slate-600
rounded-lg p-4">
                    <div className="flex items-start gap-3">
                      <AlertCircle className="w-5 h-5 text-yellow-500 flex-
shrink-0 mt-0.5" />
                      <div>
                        <p className="text-sm font-medium text-white">סכום
שהזנת: ₪{parseInt(formData.totalDebtAmount).toLocaleString('he-IL')}</p>
                        <p className="text-xs text-slate-400 mt-1">
                          אם זה לא נכון, אתה יכול לתקן זאת בכל עת
                        </p>
                      </div>
                    </div>
                  </div>
                )}
              </div>
            )}

            {/* Step 3: Additional Info */}
            {currentStep === 3 && (
```

```
              <div className="space-y-4">
                <div>
                  <label className="block text-sm font-medium text-white mb-
2">
                    האם יש הוצאה לפועל או הליך משפטי?
                  </label>
                  <Textarea
                    placeholder="לדוגמה: יש לי מכתב מעו"ד, או יש עיקול על חשבון
בנק"
                    value={formData.collectionActions}
                    onChange={(e) => handleInputChange('collectionActions',
e.target.value)}
                    className="bg-slate-700 border-slate-600 text-white
placeholder-slate-400 min-h-24"
                  />
                </div>

                <div>
                  <label className="block text-sm font-medium text-white mb-
2">
                    מידע נוסף שחשוב לנו לדעת
                  </label>
                  <Textarea
                    placeholder="לדוגמה: אני עצמאי, או אני בתהליך פשיטת רגל"
                    value={formData.additionalContext}
                    onChange={(e) => handleInputChange('additionalContext',
e.target.value)}
                    className="bg-slate-700 border-slate-600 text-white
placeholder-slate-400 min-h-24"
                  />
                </div>

                <div className="bg-blue-500/10 border border-blue-500/30
rounded-lg p-4">
                  <div className="flex items-start gap-3">
                    <CheckCircle2 className="w-5 h-5 text-blue-400 flex-
shrink-0 mt-0.5" />
                    <div>
                      <p className="text-sm font-medium text-blue-300">בסיום
תקבל:</p>
                      <ul className="text-xs text-blue-200 mt-2 space-y-1">
                        <li>✓ סיכום מצב החוב שלך</li>
                        <li>✓ המלצה על בעל מקצוע מתאים</li>
                        <li>✓ תוכנית פעולה ראשונית</li>
                      </ul>
                    </div>
```

```
                </div>
              </div>
            </div>
          )}

          {/* Action Buttons */}
          <div className="flex gap-3 pt-6 border-t border-slate-700">
            <Button
              variant="outline"
              onClick={handleBack}
              disabled={currentStep === 1 || loading}
              className="flex-1 border-slate-600 text-slate-300 hover:bg-
slate-700"
            >
              חזור
            </Button>

            {currentStep < 3 ? (
              <Button
                onClick={handleNext}
                disabled={
                  (currentStep === 1 && !formData.debtType) ||
                  (currentStep === 2 && !formData.totalDebtAmount) ||
                  loading
                }
                className="flex-1 bg-blue-600 hover:bg-blue-700 text-
white"
              >
                הבא <ChevronRight className="w-4 h-4 ml-2" />
              </Button>
            ) : (
              <Button
                onClick={handleSubmit}
                disabled={loading}
                className="flex-1 bg-green-600 hover:bg-green-700 text-
white"
              >
                {loading ? 'מעבד...' : 'סיים אבחון'}
              </Button>
            )}
          </div>
        </CardContent>
      </Card>

      {/* Step Indicators */}
      <div className="flex justify-center gap-2 mt-8">
```

```
          {[1, 2, 3].map((step) => (
            <button
              key={step}
              onClick={() => step < currentStep && setCurrentStep(step)}
              className={`w-3 h-3 rounded-full transition-all ${
                step === currentStep
                  ? 'bg-blue-500 w-8'
                  : step < currentStep
                  ? 'bg-green-500 cursor-pointer hover:bg-green-400'
                  : 'bg-slate-600'
              }`}
            />
          ))}
        </div>
      </div>
    </div>
  );
}
```

7. TODO LIST

```
# Freedom - Autonomous Debt Management Ecosystem - TODO

## Phase 1: Database Schema & Core Infrastructure
- [ ] Implement Drizzle schema for users table with roles (debtor, professional, admin)
- [ ] Create cases/debt_profiles table with severity levels and persona classification
- [ ] Build documents table with encryption metadata and audit trail
- [ ] Implement tasks table with status tracking and deadline management
- [ ] Create consent_records table for privacy law compliance (Amendment 13)
- [ ] Build audit_logs table for all sensitive operations
- [ ] Implement professional_profiles table with specializations
- [ ] Create matching_rules table for algorithm configuration
- [ ] Set up database migrations and seed initial data
- [ ] Implement AES-256 encryption helper functions

## Phase 2: Authentication & Security
- [ ] Implement OAuth 2FA flow integration
- [ ] Build WebAuthn biometric authentication (fingerprint/face)
- [ ] Create session management with secure cookies
- [ ] Implement role-based access control (RBAC) middleware
- [ ] Build consent verification system for data access
- [ ] Create audit logging for all auth events
- [ ] Implement password reset and account recovery flows
- [ ] Set up rate limiting for login attempts

## Phase 3: Backend API Layer
- [ ] Build case management procedures (create, read, update, list)
- [ ] Implement document upload/download with encryption
- [ ] Create task management procedures
- [ ] Build professional matching procedures
- [ ] Implement consent management procedures
- [ ] Create notification trigger procedures
- [ ] Build reporting and analytics procedures
- [ ] Implement audit log query procedures

## Phase 4: AI Integration Pipeline
- [ ] Implement LLM-based debt triage system (Yossi/Dana/Avi classification)
- [ ] Build document OCR and text extraction
- [ ] Implement AI document summarization
- [ ] Create AI-powered task extraction from documents
- [ ] Build professional matching algorithm
```

- [ ] Implement severity level detection
- [ ] Create risk assessment AI agent
- [ ] Build compliance checking AI agent

Phase 5: Frontend - Personal Path Dashboard
- [x] Create landing page with authentication flow
- [x] Build triage wizard (questionnaire flow) - עם UI משופר
- [x] Implement case dashboard with status overview - עם 6 כרטיסים
- [x] Create Professional Diagnosis Wizard (4-step comprehensive diagnosis)
- [x] Implement RiskCalculator with 0-200 scale and 4 risk levels
- [x] Create 5 new components for diagnosis flow
- [x] Add database schema for diagnoses table
- [x] Implement diagnosisRouter with save/retrieve procedures
- [ ] Create documents section with upload/download
- [ ] Build tasks section with reminders
- [ ] Implement notifications center
- [ ] Create professional matching UI
- [ ] Build case timeline/progress tracker
- [ ] Implement messaging interface with professionals

Phase 6: Frontend - Pro Hub Dashboard
- [ ] Create professional login and onboarding
- [ ] Build client list/management interface
- [ ] Implement case details view with full history
- [ ] Create document review interface
- [ ] Build task assignment and tracking
- [ ] Implement messaging/communication interface
- [ ] Create reporting and analytics dashboard
- [ ] Build client communication templates
- [ ] Implement billing and subscription management

Phase 7: Notification & Integration Layer
- [ ] Implement WhatsApp Business API integration
- [ ] Build Email notification system
- [ ] Create notification templates for key events
- [ ] Implement audit logging system
- [ ] Build agent orchestration hooks
- [ ] Create webhook system for external integrations
- [ ] Implement notification preferences management
- [ ] Build notification delivery tracking

Phase 8: Testing, Documentation & Deployment
- [ ] Write E2E tests for critical user flows
- [ ] Implement security testing (encryption, auth)
- [ ] Create compliance verification tests
- [ ] Build performance testing suite

```
- [ ] Write API documentation
- [ ] Create user documentation
- [ ] Build deployment scripts
- [ ] Set up monitoring and alerting
- [ ] Create disaster recovery procedures

## Core Features Status
- [ ] 2FA Authentication
- [ ] WebAuthn Biometric Support
- [ ] AI Debt Triage System
- [ ] Professional Matching Engine
- [ ] Secure Document Management (AES-256)
- [ ] OCR & Document Processing
- [ ] Dual Dashboard Interfaces
- [ ] Task & Reminder System
- [ ] WhatsApp Integration
- [ ] Email Notifications
- [ ] Consent Management (Privacy Law Compliant)
- [ ] Case Management Workflow
- [ ] Audit Logging System
- [ ] Agent Orchestration (61 Agents)
- [ ] RBAC System
- [ ] Encryption Layer

## 🐛 בעיות שדורשות תיקון (מהמשתמש):
- [x] תיקון שגיאות API (insertId undefined)
- [x] תיקון JOIN בין cases ו-debtProfiles
- [x] מילוי Dashboard בכרטיסי מצב אמיתיים
- [x] תיקון מערכת ההתחברות כמו אטו ט
- [x] הוספת כרטיס "מצב החוב שלי"
- [x] הוספת כרטיס "מה לעשות עכשיו"
- [x] הוספת כרטיס "בעלי מקצוע מחוברים"
- [x] הוספת כרטיס "מסמכים חסרים"
- [x] הוספת כרטיס "משימות קרובות"
- [x] הוספת כרטיס "התקדמות כללית"
- [x] הוספת אפשרות להוסיף חובות מרובים (Multi-Debt Support)
- [x] בנייה של Professional Diagnosis Wizard (4 שלבים)
- [x] יצירת RiskCalculator 0-200 עם scale
- [x] יצירת 5 קומפוננטות חדשות לזרימת האבחון
- [x] הוספת diagnoses table לבסיס הנתונים
- [x] חיבור diagnosisRouter לתוך appRouter

## 🎨 שיפורי UX/Design:
- [x] חיזוק כותרת Triage עם הסבר ברור
- [x] הוספת פס התקדמות (Progress Bar) בתוך Triage
- [x] הוספת דוגמאות ו-Placeholder טובים יותר
```

- [x] הוספת עזרה קטנה ליד כל שדה
- [x] הוספת Triage בסוף "תוצאה צפויה"
- [x] יותר רווח נשימה (Padding/Spacing)
- [x] צבעים לפי משמעות (ירוק/צהוב/אדום/כחול)
- [x] שפה רכה ומחזיקה (לא מאיימת)

✅ סטטוס סופי** - Phase 5 Complete:**
- ✅ 26 Tests Passing
- ✅ 0 TypeScript Errors
- ✅ Professional Diagnosis Wizard (4 steps)
- ✅ RiskCalculator (0-200 scale, 4 risk levels)
- ✅ Dashboard with 6 Feature Cards
- ✅ All Routes Connected
- ✅ Hebrew RTL Support
- ✅ Database Schema Complete
- ✅ OAuth Flow Fixed
- ✅ Build: 5.73s

Known Issues & Blockers

Notes
- All timestamps stored as UTC Unix timestamps (milliseconds)
- All sensitive data encrypted with AES-256
- All operations logged to audit_logs table
- Compliance with Israeli Privacy Protection Law Amendment 13 (2024)
- No personal data stored in logs, only operation types and outcomes

🟢 בעיות RTL (עברית-אנגלית) - בעברית!
- [x] הוספת dir="rtl" ל-HTML root
- [x] תיקון CSS RTL - text-align, margin, padding
- [x] תיקון FREEDOM positioning בכותרת
- [x] תיקון כל הקבצים - עברית-ראשי, אנגלית מוטבעת בתקשורת שלי
- [x] בדיקת כל הטקסטים בממשק

🟢 בעיות OAuth - לולאה אינסופית של התחברות
- [x] תיקון getLoginUrl - הוספת returnPath ב-state
- [x] תיקון oauth.ts - קריאת returnPath מ-state
- [x] תיקון session verification - הסרט דרישה ש-name ריק יהיה
- [x] כתיבת tests - 10 tests עברו בהצלחה
- [x] בדיקה - השרת רץ ללא שגיאות

🔧 Bug Fixes - Session 4

Issue: Unclear Debt Categories

```
- [x] הוספת הבחנה בין סוגי הלוואות אשראי:
  - [x] כרטיס אשראי
  - [x] הלוואה מחברת אשראי
  - [x] הלוואה על מסגרת אשראי (Credit Line)
- [x] הוספת חובות עירוניים לקטגוריות
- [x] עדכון ProfessionalDiagnosis עם קטגוריות חדשות
- [x] בדיקה שהשינויים עובדים בממשק
```